

Compilador de Ada em Haskell

Trabalho Prático - Compiladores

Departamento de Ciência de Computadores

Faculdade de Ciências da Universidade do Porto

Novembro 2025

Grupo 5

Flávia Queiroz - up20230959

Francisca Macedo - up202205575

Introdução

Este projeto implementa um compilador completo para a linguagem Ada, utilizando Haskell como linguagem de implementação. O compilador percorre todas as fases fundamentais de compilação, desde a análise léxica até à geração de Código MIPS executável.

Objetivos do Projeto

Fase 1: Implementar scanner (análise léxica) e parser (análise sintática)

Fase 2: Desenvolver tabela de símbolos e análise semântica

Fase 3: Gerar código intermédio (Three-Address Code)

Fase 4: Produzir código assembly MIPS executável

Aplicar conceitos teóricos de compilação em contexto prático

Demonstrar o uso de ferramentas especializadas (Alex, Happy)

Arquitetura do Compilador

O compilador está organizado em cinco componentes principais:

Pipeline de Compilação:

```
Código Ada → [Lexer] → Tokens → [Parser] → AST → [Semantic] → Symbol Table
```

↓

```
[TAC Gen] → Three-Address Code
```

↓

```
[MIPS Gen] → Código MIPS Assembly
```

Fases de Compilação

1. Análise Léxica (Lexer) - Converte código-fonte em tokens
2. Análise Sintática (Parser) - Valida a gramática e constrói a AST
3. Análise Semântica (Sematic) - Verifica tipos e constrói tabelas de símbolos
4. Geração de Código Intermédio (TAC) - Produz Three-Address Code
5. Geração de Código Final (MIPS) - Gera assembly MIPS executável

Estrutura do Projeto

Organização de Ficheiros

AST.hs

Define as estruturas de dados da Árvore de Sintaxe Abstrata, incluindo tipos para programas, statements, expressões e operadores.

Lexer.x

Especificação do analisador léxico usando Alex. Define expressões regulares para reconhecimento de:

- Palavras reservadas (**procedure**, **if**, **while**, etc.)
- Identificadores e literais
- Operadores e símbolos
- Tratamento de espaços e comentários

Parser.y

Especificação do analisador sintático usando Happy. Implementa:

- Regras gramaticais do subset de Ada
- Construção automática da AST
- Declarações de precedência e associatividade
- Mecanismos de tratamento de erros sintáticos

SymbolTable.hs

Implementa a tabela de símbolos:

- Estruturas de dados para variáveis (**VarInfo**)
- Tipos suportados (**TInt**, **TBool**)
- Operações de inserção e consulta
- Gestão de offsets de memória

Semantic.hs

Realiza análise semântica:

- Inferência de tipos de expressões
- Verificação de compatibilidade de tipos
- Detecção de variáveis não declaradas
- Construção da tabela de símbolos

TAC.hs

Gerador de código intermédio (Three-Address Code):

- Instruções TAC (operações, saltos, labels)
- Gestão de temporários e labels
- Tradução de construções de alto nível para TAC

MIPS.hs

Gerador de código assembly MIPS:

- Tradução de instruções TAC para MIPS
- Alocação de registos temporários (**\$t0 - \$t7**)
- Gestão de stack frame para variáveis
- Geração de syscalls para I/O
- Secções **.data** e **.text**

Main.hs

Módulo principal que:

- Lê o código-fonte do ficheiro de entrada
- Invoca o lexer para tokenização
- Passa tokens ao parser para análise sintática
- Exibe tokens e AST resultantes

compiler.cabal

Ficheiro de configuração para o sistema de compilação Haskell Cabal. Gere as dependências do projeto, opções do compilador e definições de compilação.

Subset de Ada Suportado

O compilador implementa suporte para as seguintes construções da linguagem Ada:

Estrutura do Programa:

- Procedimento `main` obrigatório
- Bloco `begin...end`

Expressões:

- Aritméticas: `+`, `-`, `*`, `/`
- Booleanas: `and`, `or`, `not`
- Relacionais: `=`, `/=`, `<`, `>`, `<=`, `>=`
- Literais: inteiros, booleanos (`true`, `false`)
- Variáveis e parênteses para agrupamento

Comandos (Statements):

- Atribuições: `x := expressão;`
- Condicionais: `if ... then ... else ... end if;`
- Loops: `while ... loop ... end loop;`
- I/O: `put_line(expr)` e `get_line()`

Exemplo de Programa Válido

```
procedure main is
begin
  x := 5;
  y := x + 10;
  put_line(y);
end main;
```

Configuração e Compilação

Pré-requisitos

GHC (Glasgow Haskell Compiler) ≥ 8.10

Alex ≥ 3.2 - Gerador de analisadores léxicos

Happy ≥ 1.19 - Gerador de parsers LALR(1)

Cabal para gerir as dependências

MIPS (MIPS Assembler and Runtime Simulator) - Para executar o código gerado

Compilação Manual

```
# Gerar Lexer.hs a partir de Lexer.x
alex Lexer.x

# Gerar Parser.hs a partir de Parser.y
happy Parser.y

# Compilar todos os módulos Haskell
ghc --make -o compiler Main.hs

# Executar o compilador
./compiler <ficheiro.ada>

ou
cabal clean
cabal run

# Executar o compilador
./compiler <ficheiro.ada>
```

Localização dos Ficheiros Gerados

Os ficheiros **Lexer.hs** e **Parser.hs** são gerados automaticamente no diretório do projeto. Estes ficheiros contêm código Haskell otimizado produzido pelo Alex e Happy, respetivamente.

Execução do Compilador

Modo de Uso

```
./compiler <ficheiro.ada>
```

Exemplo

```
./compiler harder_tests/test9.ada

=== TOKENS (Lexer) ===
TOK_PROCEDURE
TOK_MAIN
TOK_IS
TOK_BEGIN
TOK_ID "a"
TOK_ASSIGN
TOK_INT 48
TOK_SEMI
TOK_ID "b"
TOK_ASSIGN
```

```
TOK_INT 18
TOK_SEMI
TOK_WHILE
TOK_ID "b"
TOK_NEQ
TOK_INT 0
TOK_LOOP
TOK_ID "temp"
TOK_ASSIGN
TOK_ID "b"
TOK_SEMI
TOK_ID "b"
TOK_ASSIGN
TOK_ID "a"
TOK_MINUS
TOK_LPAREN
TOK_ID "a"
TOK_DIV
TOK_ID "b"
TOK_RPAREN
TOK_MUL
TOK_ID "b"
TOK_SEMI
TOK_ID "a"
TOK_ASSIGN
TOK_ID "temp"
TOK_SEMI
TOK_END
TOK_LOOP
TOK_SEMI
TOK_PUT_LINE
TOK_LPAREN
TOK_ID "a"
TOK_RPAREN
TOK_SEMI
TOK_END
TOK_MAIN
TOK_SEMI
TOK_EOF
```

=== Árvore de Sintaxe Abstrata (AST) ===

```
ProgramMain [SAssign "a" (EInt 48),SAssign "b" (EInt
18),SWhile (EBinOp Neq (EVar "b") (EInt 0)) (SSeq [SAssign
"temp" (EVar "b"),SAssign "b" (EBinOp Sub (EVar "a") (EBinOp
Mul (EBinOp Div (EVar "a") (EVar "b"))) (EVar "b"))),SAssign
"a" (EVar "temp")]],SPutLine (EVar "a")]
```

=== Análise Semântica ===

Sem erros semânticos

=== Tabela de Símbolos ===

```
VarInfo {varName = "a", varType = TInt, varInitialized =
True, varLocation = 0}
VarInfo {varName = "b", varType = TInt, varInitialized =
True, varLocation = 4}
VarInfo {varName = "temp", varType = TInt, varInitialized =
True, varLocation = 8}
```

=== THREE-ADDRESS CODE ===

```
    t0 := 48
    a  := t0
    t1 := 18
    b  := t1
L_while_start_0:
    t2 := b
    t3 := 0
    t4 := t2 != t3
    t5 := not t4
    if t5 goto L_while_end_1
    t6 := b
    temp := t6
    t7 := a
    t8 := a
    t9 := b
    t10 := t8 / t9
    t11 := b
    t12 := t10 * t11
    t13 := t7 - t12
    b := t13
    t14 := temp
    a := t14
    goto L_while_start_0
L_while_end_1:
    t15 := a
    print t15
```

=== Geração de código MIPS ===

Código MIPS guardado em: harder_tests/test9.asm

Compilado Corretamente!

Correr com: java -jar Mars4_5.jar harder_tests/test9.asm

java -jar Mars4_5.jar harder_tests/test9.asm

6

cat harder_tests/test9.asm

.data

newline: .asciiz "\n"

.text

.globl main

main:

 # Prólogo: setup stack frame

 addi \$sp, \$sp, -100 # Reservar 100 bytes na stack

 li \$t0, 48

 move \$t1, \$t0

 sw \$t1, 0(\$sp)

 li \$t2, 18

 move \$t3, \$t2

 sw \$t3, 4(\$sp)

L_while_start_0:

 lw \$t4, 4(\$sp)

 li \$t5, 0

 sne \$t6, \$t4, \$t5

 seq \$t7, \$t6, \$zero

 bnez \$t7, L_while_end_1

 lw \$t0, 4(\$sp)

 move \$t1, \$t0

 sw \$t1, 8(\$sp)

 lw \$t2, 0(\$sp)

 lw \$t3, 0(\$sp)

 lw \$t4, 4(\$sp)

 div \$t3, \$t4

 mflo \$t5

 lw \$t6, 4(\$sp)

 mul \$t7, \$t5, \$t6

 sub \$t0, \$t2, \$t7

 move \$t3, \$t0

 sw \$t3, 4(\$sp)

 lw \$t1, 8(\$sp)

 sw \$t1, 0(\$sp)

 j L_while_start_0

L_while_end_1:

 lw \$t2, 0(\$sp)

 # print_int

 move \$a0, \$t2


```
li $v0, 1
syscall
# print newline
li $v0, 4
la $a0, newline
syscall

# Epílogo: exit
li $v0, 10          # syscall 10 = exit
syscall
```

Ficheiro Assembly Gerado

O compilador gera um ficheiro .asm com o mesmo nome no ficheiro fonte

O compilador produz duas secções de output:

1. **Lista de Tokens** - Resultado da análise léxica
 2. **Árvore de Sintaxe Abstrata (AST)** - Resultado da análise sintática
 3. **Lista de Erros Semânticos (Semantic.hs)** - Resultado da análise semântica
 4. **Lista de Símbolos** - resultado da Tabela de Símbolos
 5. **Código Intermédio** - resultado TAC Three-Address Code
 6. **Código Executável** - ficheiro em Código MIPS
-

Detalhes de Implementação

Análise Léxica (Lexer.x)

O lexer é responsável pela primeira fase de compilação: transformar uma sequência de caracteres numa sequência de tokens.

Análise Sintática (Parser.y)

O parser implementa um analisador sintático LALR(1) que verifica se a sequência de tokens respeita a gramática da linguagem e constrói a AST correspondente.

Árvore de Sintaxe Abstrata (AST.hs)

A AST é definida usando tipos algébricos de Haskell, proporcionando uma representação estruturada e type-safe do programa.

Análise Semântica (Semantic.hs)

Verifica:

- **Inferência de tipos:** Determina automaticamente o tipo de cada variável
- **Compatibilidade de tipos:** Garante operações válidas (ex: `Int + Int`, `Bool and Bool`)
- **Variáveis não declaradas:** Detecta uso de variáveis antes da atribuição
- **Operadores:** Verifica que operadores são aplicados a tipos corretos

Tipos de Erros Semânticos

```
data SemanticError
  = UndeclaredVariable String
  | TypeMismatch String VarType VarType
  | UninitializedVariable String
```

Código Intermédio (TAC)

O TAC simplifica a geração de código final através de:

- **Instruções de 3 endereços:** `t1 := t2 op t3`
- **Temporários:** Variáveis virtuais (`t0`, `t1`, ...)
- **Labels:** Para controlo de fluxo (`L_while_start_0`, ...)
- **Operações simples:** Cada instrução realiza apenas uma operação

Geração de Código MIPS

Convenções de Registos:

- **\$t0-\$t7** : Registos temporários (alocação cíclica)
- **\$sp**: Stack pointer (aponta para topo da stack)
- **\$a0**: Argumento para syscalls
- **\$v0**: Código de syscall
- **\$zero**: Constante 0

Gestão de Memória:

- Variáveis são armazenadas na stack
- Cada variável tem um **offset** fixo relativo a **\$sp**
- Temporários são mantidos em registos
- Stack frame de **100 bytes** por padrão

Syscalls Utilizadas:

- 1: Print Integer (`put_line`)
- 4: Print string (newline)
- 5: Read integer (`get_line`)
- 10: Exit program

Testes

O compilador foi validado com 7 programas de teste que cobrem as funcionalidades implementadas:

Teste	Funcionalidade
test1.ada	Output básico / Estrutura mínima
test2.ada	Atribuições de variáveis e Aritmética
test3.ada	Condicional simples (if-then)
test4.ada	Condicional composta (if-then-else)
test5.ada	Ciclos de repetição (while loop)
test6.ada	Leitura de input (get_line)
test7.ada	Operações booleanas e Comparações

Testes Extra

com teor mais complexo e com o objetivo de melhorar o nosso compilador em casos menos básicos

test1.ada	Aritmética Complexa (Precedência e Negativos)
test2.ada	Expressões Booleanas Complexas (AND, OR, NOT)
test3.ada	If-Then-Else Aninhados (Máximo de 3 números)
test4.ada	While Loops Aninhados (Soma Triangular)
test5.ada	Tabuada (Múltiplos de 7)
test6.ada	Fatorial (Cálculo iterativo)
test7.ada	Sequência de Fibonacci
test8.ada	Verificação de Número Primo
test9.ada	Máximo Divisor Comum (GCD - Euclides)
test10.ada	Soma de Dígitos de um Inteiro
test11.ada	Potência (Exponenciação)
test12.ada	Número Perfeito (Soma de divisores)
test13.ada	Inversão de Número (Reversão de dígitos)
test14.ada	Palíndromo Numérico

test15.ada	Contagem de Números Pares
test16.ada	Sequência de Collatz ($3n+1$)
test17.ada	Ordenação Simples (Bubble Sort parcial)
test18.ada	Cálculo de Média (Simulação de input)
test19.ada	Sistema de Classificação (Ranges de valores)
test20.ada	Calculadora Interativa (4 operações)

Tratamento de Erros

Erros Léxicos

O lexer ignora caracteres inválidos e continua o processamento. Caracteres não reconhecidos são reportados mas não interrompem a análise.

Erros Sintáticos

O parser fornece mensagens de erro indicando:

- Posição aproximada do erro (10 primeiros tokens seguintes)
- Tipo de erro detetado

Erros Semânticos

- Compilação interrompida
- Não gera código TAC ou MIPS

Lista todos os erros semânticos encontrados antes de parar

Permite Múltiplos erros serem reportados numa única execução

Conclusão

O projeto demonstra a implementação de um compilador simplificado de Ada em Haskell, cobrindo todas as fases fundamentais de compilação, análise léxica e sintática, análise semântica com tabela de símbolos, geração de código intermédio (TAC) e geração de código assembly MIPS executável.

O uso de ferramentas especializadas (Alex e Happy) e a natureza funcional do Haskell permitiram construir uma pipeline de compilação robusta, modular e bem estruturada. O código MIPS gerado é funcional e pode ser executado no simulador MARS, demonstrando que todas as fases do compilador funcionam corretamente em conjunto. Este trabalho proporciona uma compreensão profunda do funcionamento interno de compiladores modernos e das técnicas fundamentais de construção de compiladores.